Rule Substitution and Chaining

Idea: make the rule's conclusion match the current goal. Substitute those bindings into its inputs, then use one input as the next backward goal.

Inputs: a target statement, a valid forward rule sequence, and an object pool. Each backward template specifies a conclusion, one input to continue from, and any supporting premises.

Algorithm 1: Instantiate rules backward to construct a connected chain

```
goal ← target statement
premises ← empty list
for rule in reverse(rule sequence) do
    template ← the rule's backward pattern
    bindings ← match template.conclusion to goal
    if the match is invalid then
        return construction failure
    end
    Fill remaining variables with suitable objects or spatial atoms
    Keep existing bindings fixed and satisfy the rule's constraints
    input, support ← substitute bindings into the template
    Append support to premises
    goal ← input
end
Append the final goal to premises
return premises that establish the original target
```

Example: $L = 2$. Target: `NE(A, B)`. Forward rule sequence: **modus ponens, then transitivity**. Construction applies these rules in reverse:

1. **Transitivity:** match conclusion `D(x, z)` to `NE(A, B)`. Bind `D = NE`, `x = A`, `z = B`; choose `y = M`, distinct from A and B. The new goal is `NE(A, M)`; add support `NE(M, B)`.
2. **Modus ponens:** bind its conclusion `S = NE(A, M)`. Choose `P = East(C, D)`. The new goal is P; add support `P -> NE(A, M)`.

Generated premises:

```
East(C, D)
East(C, D) -> NE(A, M)
NE(M, B)
```

Forward solution:

1. Modus ponens derives `NE(A, M)`.
2. Transitivity combines it with `NE(M, B)` to derive `NE(A, B)`.

There are **two dependent rule applications** but three premises. The certificate must confirm the realised length is 2. The NE direction is inherited from the goal, not independently randomised on each transitivity edge.

Object variables receive object names; Boolean variables receive whole formulas. The generator subsequently checks consistency, answers, and the realised proof. Matching and substitution describe the method conceptually: the current code implements these patterns as five explicit rule cases, not a general unifier.

Two-Rule Chains

Read each pair in **forward proof order**: the row is the first rule, and the column is the next rule. Backward construction traverses the pair in reverse. The current constructor supports **23 of the 25 ordered pairs**:

First / Next	MP	IFF	DJE	DN	TRC
MP	Yes	Yes	Yes	Yes	Yes
IFF	Yes	Yes	Yes	Yes	Yes
DJE	Yes	Yes	Yes	Yes	Yes
DN	Yes	Yes	Yes	Yes	Yes
TRC	Yes	Yes	No	No	Yes

MP: modus ponens; **IFF:** equivalence elimination; **DJE:** disjunctive syllogism; **DN:** double-negation elimination; **TRC:** transitivity.

Why two pairs are blocked. TRC produces an exact positive spatial atom. The current backward templates for DJE and DN require a negated or double-negated input from the preceding step. Therefore, TRC cannot immediately feed either template. This is a restriction of our construction grammar, not of logic in general.

Examples. P, Q, and S stand for spatial formulas.

```
MP -> DN
Premises: P; P -> NOT NOT S
Steps:    derive NOT NOT S; eliminate double negation to derive S.

DJE -> MP
Premises: P OR Q; NOT P; Q -> S
Steps:    derive Q; use Q -> S to derive S.

TRC -> MP
Premises: NE(A,M); NE(M,B); NE(A,B) -> S
Steps:    derive NE(A,B); use the implication to derive S.
```

Admission still applies. A supported pair means the constructor can attempt that chain; the certificate must still confirm the realised length is 2. Training policies restrict the allowed transitions further, reserving some pairs for held-out evaluation. Query and answer-set policies can impose additional restrictions.